Tesla Energy Interview: 30-Second Stories

Oct 31, 2026 · @Amogh

Stories for the Tesla Energy Product Engineer (Industrial Storage, Diagnostics) interview. Each runs about 30 seconds spoken (roughly 75 to 85 words), with the Tesla tie-in built into the ending. Bracketed items are details to fill in before rehearsing.

Jump to: Questions to ask the recruiter

Index

1. Tell me about yourself
2. Why Tesla?
3. Tell me about a root cause you found
4. Tell me about something you automated
5. Tell me about working with messy data
6. How would you evaluate a firmware update?
7. Do you have battery experience?
8. Tell me about a challenge or failure
9. What are your strengths?
10. What is your biggest weakness?
11. Questions to ask the recruiter
12. Timing tip

Tell me about yourself (30 seconds)

> I'm Amogh, finishing my Master's in Data Science at RIT this December. I use data to keep physical equipment running. At Penske I worked with truck telemetry and found the root cause of recurring battery failures across about 3,000 trucks. At Wegmans I built automated leak detection for refrigeration across about 110 stores. This role is the same work on energy storage: find failures in fleet telemetry and automate detecting them. I'm available to start in December.

Why Tesla? (45 seconds)

> Three things: the problem, the leverage and the culture.
>
> First, the problem. Storage is what turns renewables into reliable power, and Tesla is deploying it faster than almost anyone, with a new Texas factory built for 50 gigawatt-hours a year and Megapack 3 ramping up. A fleet growing that fast, on a new hardware generation, is the hardest version of the reliability problem. That's where I want to be.
>
> Second, the leverage. Tesla designs the hardware, writes the firmware and gets data from every unit. In my past work, I could find a problem in the data but often couldn't change the equipment. Here, the team that finds a failure can drive the fix, whether that's a firmware update or a design change.
>
> Third, the culture. Tesla solves problems from first principles and owns them end to end, fast. That's how I work. At Penske the default fix for dead batteries was a new battery. I asked why they kept dying and traced it to the liftgate. I want to work with people who think that way.

Avoid: talking about Elon Musk, the stock or the cars, and "I've always loved Tesla."

Tell me about a root cause you found (Story 1: Penske battery failures)

Use for: root cause, hardware diagnosis, electronic systems

> At Penske, trucks kept breaking down on the road with dead batteries, which meant road calls and unsatisfied customers. The usual fix was a new battery, but the same trucks kept coming back. I dug into the free-text complaints and traced it to the hydraulic liftgate draining the battery faster than the truck could recharge it. I validated that across about 3,000 trucks **[by comparing trucks with and without liftgates]**.
>
> At Tesla I'd ask the same question: not just which part failed, but what was stressing it.

Tell me about something you automated (Story 2: Wegmans refrigerant leak detection)

Use for: automated detection, telemetry, catching problems early

> At Wegmans, refrigeration alarms across about 110 stores only fired once a system had already lost a lot of refrigerant. Weight should stay flat in a sealed system, so I built a pipeline in Microsoft Fabric that measures each system's loss rate in pounds per week. It detects technician top-offs in the signal so they can't hide a leak. **[Result: one leak it caught early.]**
>
> Battery fleets work the same way. The real failure is a slow trend under the daily swings, like capacity fade or a drifting cell.

Tell me about working with messy data (Story 3: NGAFID aviation research)

Use for: messy data, ML and technical depth, characterizing failure modes at scale

> In my FAA-funded research, aircraft maintenance records were free text, so failures couldn't be linked to sensor data. I joined over 350K work orders to flight data and built a hybrid classifier that sorted them into 51 failure categories at over 98% accuracy. Then I aligned each failure with sensor data by flight phase.
>
> Tesla's fleet has the same two halves: unit telemetry and field service records. Linking them is how you find a failure's fingerprint.

How would you evaluate a firmware update? (Story 4: Penske firmware A/B testing)

Use for: firmware failures, evaluating the impact of changes, reliability KPIs

> At Penske, OEM firmware updates reached the fleet in stages, so I evaluated them like an A/B test. I compared trucks on the new version with similar trucks still on the old one, on reliability KPIs like **[fault code rate per 1,000 miles, road calls and repeat repairs]**. I tracked real outcomes, not just fault codes, because an update can change what gets flagged without changing what breaks. **[Result.]**
>
> Megapack firmware ships over the air in waves, so I'd test it the same way at Tesla: one primary KPI like the targeted trip rate per unit-day, guardrails like availability and cell temperatures, and a fair comparison group.

Confirm before using: which KPIs you actually tracked, how trucks got updated (shop visit, OTA or pilot group) and what you found.

Do you have battery experience?

> My battery experience is on the truck side. At Penske I built a dashboard for CBC, the battery servicing operation, tracking how many batteries tested good enough to recharge versus how many had to be replaced, along with recharge time by state. Separately, I traced recurring on-road battery failures to tailgate power draw. It's lead-acid at truck scale, not lithium-ion at grid scale, but the core question is the same: is the battery just discharged, or is it actually degraded?
>
> On a Megapack fleet that's the state of charge versus state of health question. Getting it wrong means either replacing good hardware or missing a real failure.

Confirm before using: what the CBC team used the dashboard for, such as spotting locations or truck types with high replacement rates.

Tell me about a challenge or failure

Use for: failure, mistake, difficult challenge, ownership

> When I first built the refrigerant leak detection at Wegmans, it flagged far too many systems, **[roughly X in the first run]**. If the refrigeration team had chased those, they'd have stopped trusting the alerts.
>
> I didn't send anything out until I understood why. Going through the flagged systems, I found two causes: technician top-offs were distorting the trend, and sensor glitches were creating sudden drops that weren't real leaks. So I detected top-offs in the signal, calculated the loss rate only between them and required a drop to persist before counting it. That cut the alerts to **[Y systems]**, and **[the ones I checked with the refrigeration team were real leaks]**.
>
> Since then I validate against real outcomes before launching anything. At Tesla, a detection that fires on healthy Megapacks wastes field service time and erodes trust, so I'd validate every new detection against confirmed failures before rolling it out fleet-wide.

Be ready for: how many false alarms before and after the fix, how you knew the remaining flags were real, and why you didn't just raise the threshold (it would have delayed catching real leaks).

Confirm before using: that the early version really over-flagged, and that these were the actual causes.

What are your strengths? (45 seconds)

> I'd say three things.
>
> First, I turn technical findings into action. A root cause only matters if the right people act on it. At Wegmans I translate contractor and asset data into KPIs that Area Managers use for cost decisions. At Penske I built dashboards that replaced the team's manual Excel reports for about 20 users. I keep it short and accurate: what's happening, how sure I am and what to do next.
>
> Second, I move fast and own problems end to end. At Penske the default fix for dead batteries was a new battery. I went into the complaint data, found the liftgate power draw and validated it across about 3,000 trucks. At Wegmans I took leak detection from a vague problem to an automated pipeline across about 110 stores.
>
> Third, I adapt quickly to new systems. I've gone from trucks to aircraft to refrigeration, and each time I learned the equipment before trusting its data. That's how I'd ramp up on Megapack.

What is your biggest weakness? (30 seconds)

> I tend to try to solve problems on my own for too long before asking for help. When I started working with refrigeration data at Wegmans, I spent **[a couple of days]** trying to figure out why some systems' readings looked off from the data alone. A 15-minute conversation with a refrigeration technician answered it. Now I timebox it. If I'm stuck for **[an hour or two]**, I go to the person who knows the equipment, with what I've tried and a specific question. In a role like this, where the answer often sits with a firmware or hardware engineer, that habit matters even more.

Confirm before using: swap in a real moment when you spent too long alone on something a colleague could have answered quickly.

Questions to ask the recruiter

Ask the first three in this order. Use the fourth if there's extra time.

1. "What does the timeline look like for the rest of the process, and when are you hoping to have someone start?"

2. "What does a strong first six months look like for someone in this role?"
3. "Is there anything in my background you'd like me to expand on for the hiring manager?" A good closer, because it surfaces any doubts while you can still address them.
4. "What will the next round focus on, and how can I best prepare?"

Timing tip

Read each one aloud with a timer. If one runs past 35 seconds, trim the setup, not the tie-in. The last sentence is what the interviewer will remember.